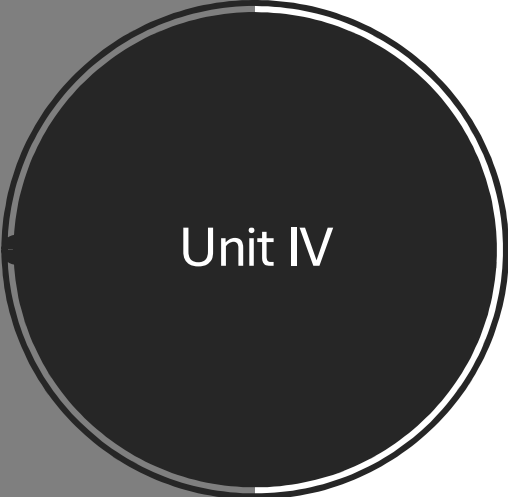




Unit IV

Concept of Clustering in ML

K- Means Clustering Algorithm

DBSCAN Algorithm

Hierarchical Clustering

Evaluation of Clustering Models

Introduction to Dimensionality Reduction

Principal Component Analysis

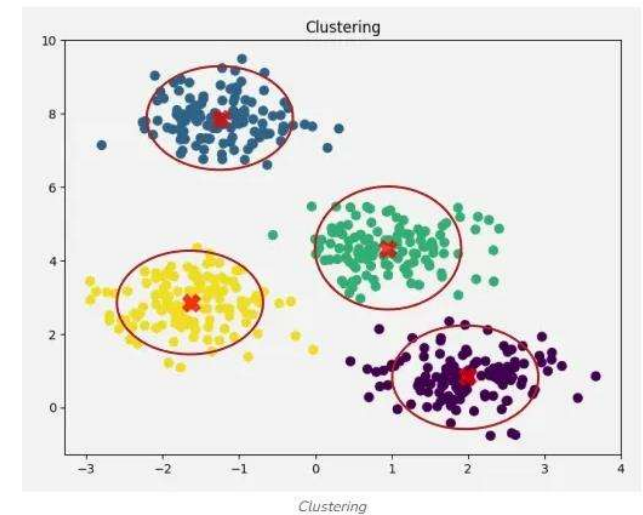
Linear Discriminant Analysis

Feature Selection vs Feature Extraction

Real- World Applications of Unsupervised Learning

Clustering

- **Definition:**
Clustering is an *unsupervised learning* technique that groups similar data points into clusters based on similarity, without labeled data.
The goal is to ensure that data points within the same cluster are more similar to each other than to those in different clusters.
- **Goal:**
Discover natural patterns and hidden structures in data without predefined categories.
- **How It Works:**
Data points are grouped using similarity or distance measures like *Euclidean* or *Cosine distance*.
- **Example:**
In customer data, clustering can group buyers with similar shopping habits for targeted marketing or personalized recommendations.
- **Types of Clustering:**
- **Hard Clustering:** Each data point belongs strictly to one cluster.
 - Example: Customer belongs only to Cluster 1 or 2.
 - Used for: Market segmentation, document grouping.
- **Soft Clustering:** Data points can belong to multiple clusters with probabilities.
 - Example: 70% in Cluster 1, 30% in Cluster 2.
 - Used for: Customer personas, medical diagnosis.



Clustering Methods & Applications

1. Centroid-Based (Partitioning)

- Algorithms: *K-Means*, *K-Medoids*
- **Pros:** Fast, scalable, simple
- **Cons:** Needs number of clusters, sensitive to outliers

2. Density-Based

- Algorithms: *DBSCAN*, *OPTICS*
- **Pros:** Detects arbitrary shapes, handles noise
- **Cons:** Parameter selection is difficult

3. Connectivity-Based (Hierarchical)

- Algorithms: *Agglomerative*, *Divisive*
- **Pros:** No need to predefine cluster count
- **Cons:** Computationally heavy

Recommendation Systems

Market Basket Analysis

4. Distribution-Based

- Algorithm: *Gaussian Mixture Model (GMM)*
- **Pros:** Flexible cluster shapes, probabilistic
- **Cons:** Needs number of components, sensitive to initialization

5. Fuzzy Clustering

- Algorithm: *Fuzzy C-Means*
- **Pros:** Handles overlap, captures ambiguity
- **Cons:** More computationally expensive

Use Cases:

Customer Segmentation Anomaly Detection Image Segmentation

Data Point	Hard Clustering	Soft Clustering
A	Cluster 1	Cluster 1: 0.91, Cluster 2: 0.09
B	Cluster 2	Cluster 1: 0.30, Cluster 2: 0.70
C	Cluster 3	Cluster 1: 0.17, Cluster 2: 0.83
D	Cluster 4	Cluster 1: 1.00, Cluster 2: 0.00

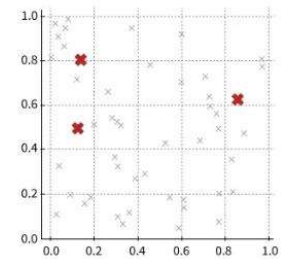
K-Means Clustering – Introduction & Working

- **Definition:**
K-Means is an unsupervised machine learning algorithm that groups unlabeled data into **k clusters** based on similarity. Each cluster is represented by its **centroid**, and data points are assigned to the nearest centroid.
- **Goal:**
To partition data into clusters such that points within a cluster are more similar to each other than to those in other clusters.
- **Example:**
An online store groups customers into segments like “Budget Shoppers,” “Frequent Buyers,” and “Big Spenders.”

Choose Initial Centroids

Centroids are randomly chosen from the data points. These represent the initial cluster centers.

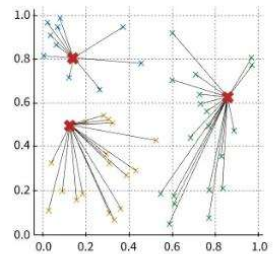
✱ Centroids ✕ Data Points



Assign Points to Nearest Centroid

Each point is assigned to the nearest centroid, forming clusters

✱ Centroids ✕ Cluster 1
✕ Cluster 2 ✕ Cluster 3



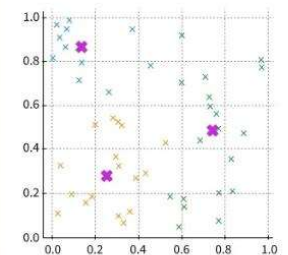
How K-Means Works:

- **Initialization:** Randomly select k cluster centroids.
- **Assignment:** Assign each data point to its nearest centroid.
- **Update:** Recalculate centroids as mean of assigned points.
- **Repeat:** Continue until centroids stabilize.
- **Selecting Optimal K:**
 - Use the **Elbow Method** — plot *Within Cluster Sum of Squares (WCSS)* vs k and find the point where adding clusters doesn't improve performance significantly.
- **Why Use K-Means?**
 - Simple & Fast
 - Effective for large datasets
 - Widely used for segmentation, anomaly detection & image compression

Update Centroids

Centroids are recalculated as the mean of the points in each cluster

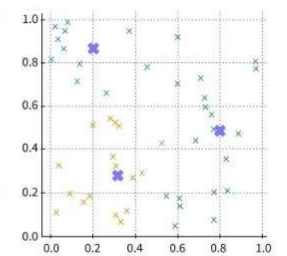
- ✱ New Centroids
- ✕ Cluster 1
- ✕ Cluster 2
- ✕ Cluster 3



Repeat Until Convergence

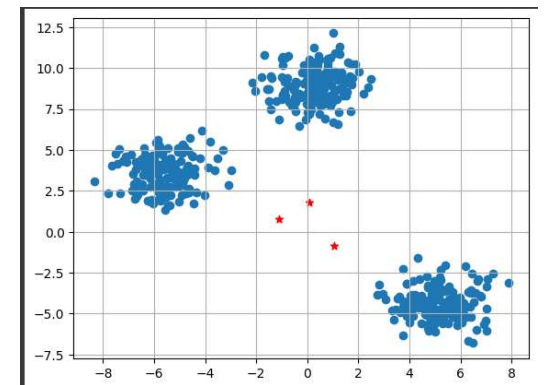
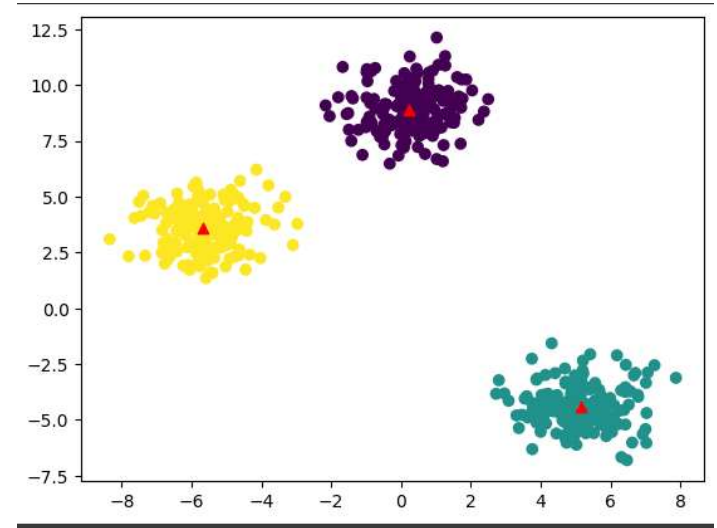
This process repeats until the centroids stabilize and do not move further.

- ✱ Final Centroids
- ✕ Cluster 1
- ✕ Cluster 2
- ✕ Cluster 3



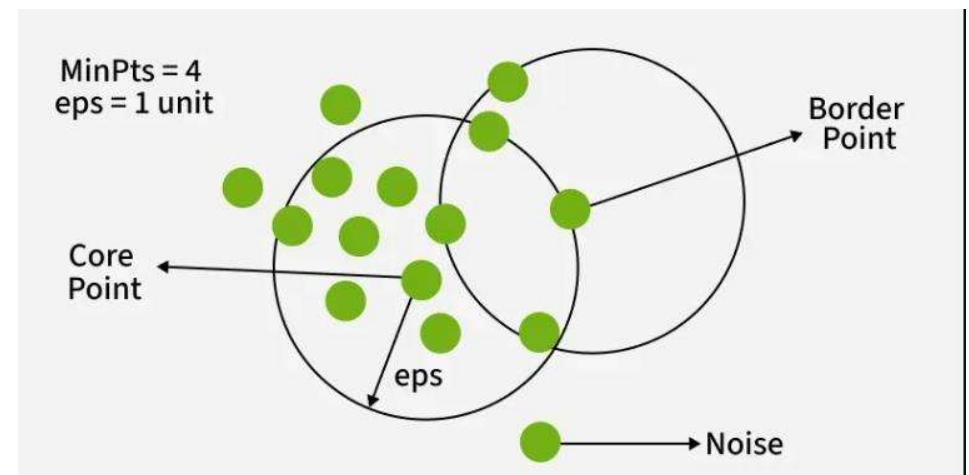
K Means Clustering - Applications

- **Applications:**
- **Customer Segmentation** – grouping users by purchase patterns
- **Image Compression** – reducing image colors using cluster centroids
- **Anomaly Detection** – identifying outliers that don't fit any cluster
- **Document Clustering** – grouping articles or news by similarity
- **Challenges:**
 - **Choosing the right number of clusters (k)**
 - Sensitive to initial centroid positions
 - Struggles with non-spherical clusters
 - Affected by outliers



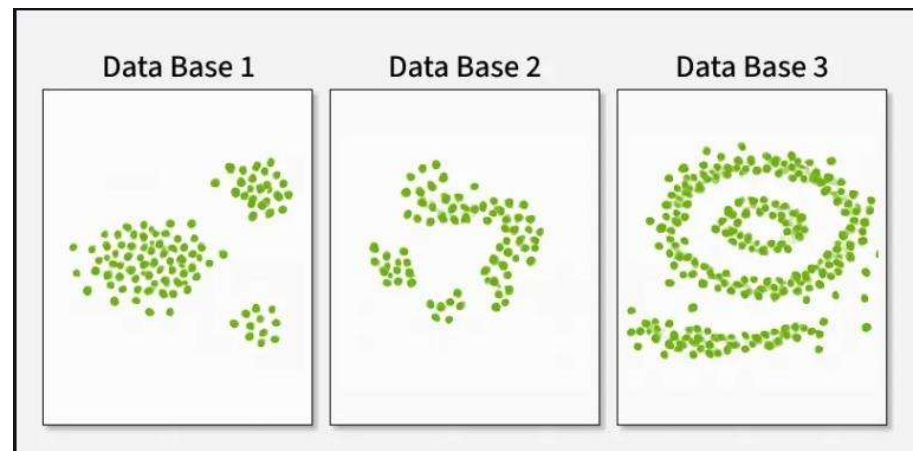
DBSCAN Clustering – Overview & Working

- **Definition:**
DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an **unsupervised clustering algorithm** that groups closely packed points and identifies outliers as noise. It is ideal for datasets with **arbitrary-shaped clusters** and **noise**.
- **Key Advantages:**
Detects non-spherical clusters
Handles noise and outliers effectively
No need to predefine number of clusters (*unlike K-Means*)
- **Main Parameters:**
- **eps (ϵ):** Neighborhood radius around each point
- **MinPts:** Minimum number of points required to form a dense region
(Rule: $MinPts \geq D + 1$, where D = number of dimensions)



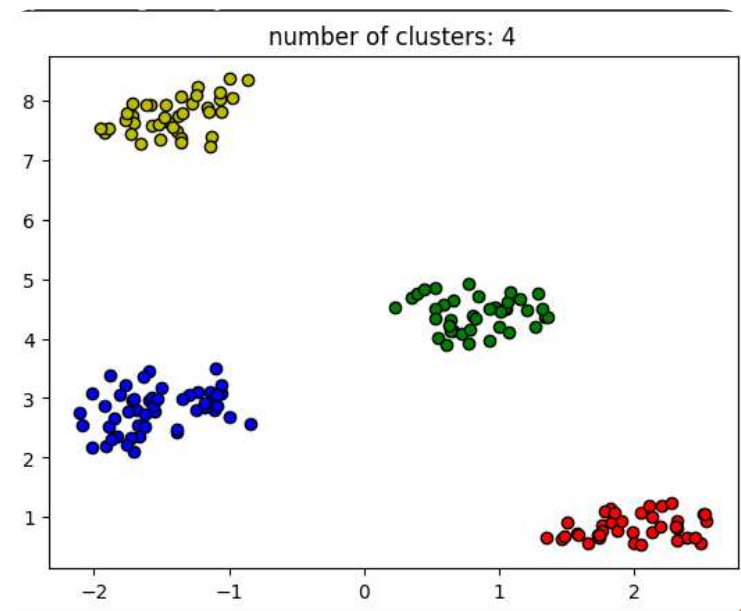
DBSCAN - Working

- **How DBSCAN Works:**
 - **Identify core points** (dense regions with $\geq$ MinPts neighbors within ϵ).
 - Connect density-reachable points to form clusters.
Label remaining points as **noise**.
- **Types of Points:**
 - **Core Points:** Have enough neighbors
 - **Border Points:** Close to core points but not dense enough
 - **Noise Points:** Do not belong to any cluster



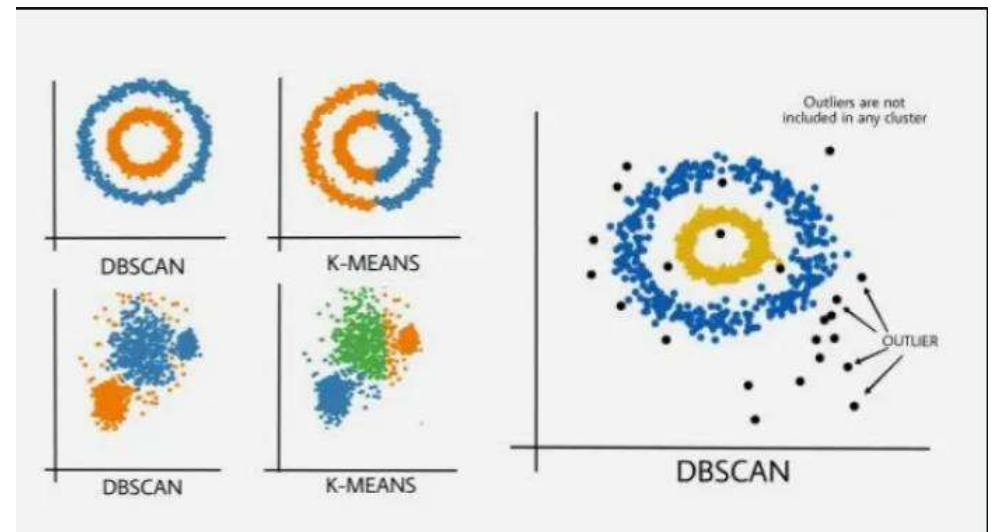
Implementation, Evaluation & Comparison

- **Evaluation Metrics:**
- **Silhouette Score (-1 to 1):** Measures cluster separation; closer to 1 is better.
- **Adjusted Rand Index (0 to 1):** Measures similarity between predicted and true labels.
- **DBSCAN vs K-Means:**
- DBSCAN does **not** require specifying number of clusters.
- Works well with **irregular shapes** and **noise**.
- K-Means assumes **spherical clusters** and is **sensitive to outliers**.
- **When to Use DBSCAN:**
- Unknown number of clusters
- Non-linear cluster shapes
- Data with outliers or variable density



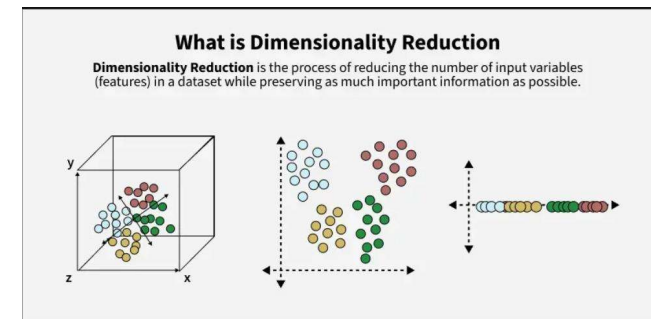
DBSCAN vs K Means

DBSCAN	K-Means
In DBSCAN we need not specify the number of clusters.	It is very sensitive to the number of clusters so it need to specified
Clusters formed in DBSCAN can be of any arbitrary shape.	Clusters formed are spherical or convex in shape
It can work well with datasets having noise and outliers	It does not work well with outliers data. Outliers can skew the clusters in K-Means to a very large extent.
In DBSCAN two parameters are required for training the Model	In K-Means only one parameter is required is for training the model



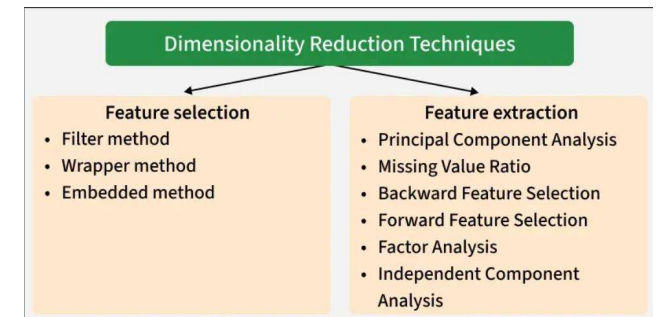
Introduction to Dimensionality Reduction

- **Definition:**
Dimensionality Reduction is a technique to reduce the number of features in a dataset while retaining essential information. It simplifies high-dimensional data into fewer dimensions for **faster computation, easier visualization, and better model performance.**
- **Why It's Needed:**
 - High-dimensional data → slow computation & risk of overfitting
 - Reduces redundancy and noise
 - Makes visualization & analysis easier
- **Example:**
In predicting house prices, using too many features (like flooring, lighting, etc.) increases complexity. Reducing dimensions keeps only the most relevant ones (bedrooms, area, location).
- **How It Works:**
 - Identifies which features carry the most variance/information
 - Projects data from higher → lower dimensions while preserving structure
- **Benefits:**
Faster computation
Easier visualization
Reduced overfitting
- **Limitation:**
Some data loss & reduced accuracy possible



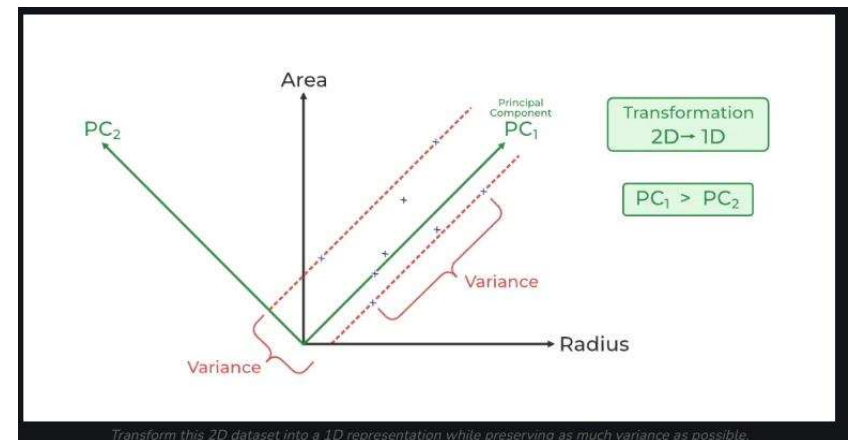
Techniques & Use Cases

- **Main Categories:**
- **Feature Selection** – Selects the most relevant original features
 - *Methods:* Filter, Wrapper, Embedded
 - *Examples:* Random Forest, Forward/Backward Selection
- **Feature Extraction** – Transforms data into new features
 - *Techniques:*
 - **PCA (Principal Component Analysis)** – Converts correlated variables to uncorrelated components
 - **Factor & Independent Component Analysis (ICA)** – Finds patterns or independent features
 - **Real-World Applications:**
- **Gene Expression Analysis:** Identify key genes faster
- **Text Categorization:** Reduce word features for classification
- **Image Retrieval:** Use fewer visual features for faster search
- **Intrusion Detection:** Select optimal features for cybersecurity models
- **Summary:**
Dimensionality Reduction =
Simpler models
Faster learning
Better generalization



Principal Component Analysis (PCA) – Overview

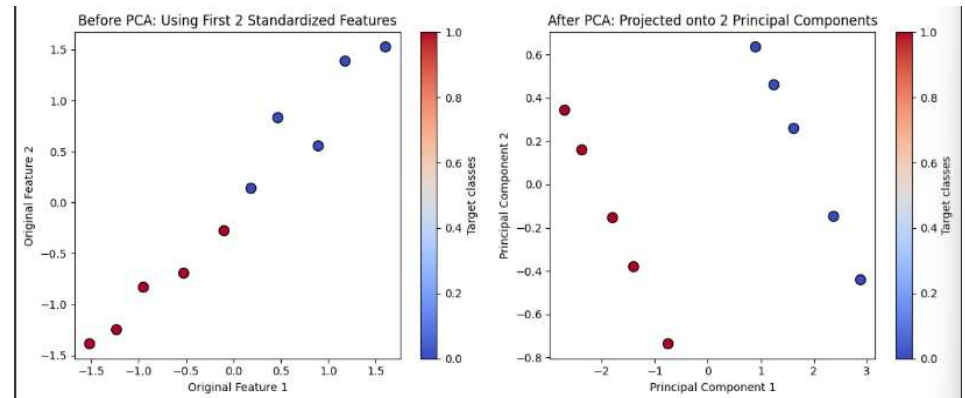
- **Definition:**
PCA is a **dimensionality reduction technique** that transforms high-dimensional data into fewer variables (called *principal components*) while preserving most of the important information.
- **Purpose:**
 - Simplifies complex datasets.
 - Removes redundancy and correlation among features.
 - Improves computational efficiency and visualization.
- **How PCA Works (Steps):**
 - **Standardize the Data:** Convert all features to have mean = 0 and std = 1.
 - **Compute Covariance Matrix:** Shows how variables vary together.
 - **Find Eigenvalues & Eigenvectors:** Identify new axes (principal components) where data varies most.
 - **Select Top Components:** Keep components that capture the highest variance.
 - **Transform Data:** Project original data onto these new axes.
- **Key Concept:**
 - PC_1 captures the maximum variance (most information).
 - PC_2 is orthogonal to PC_1 and captures the next most variance.



PCA

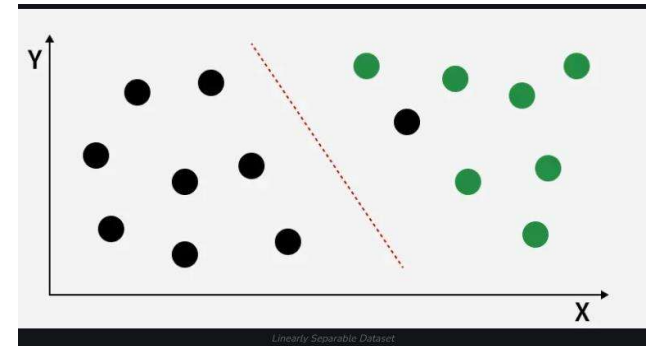
Implementation

- **Dataset:**
Features → Height, Weight, Age
Target → Gender (1 = Male, 0 = Female)
- **Advantages:**
 - Reduces dimensionality
 - Removes multicollinearity
 - Improves speed & visualization
- **Limitations:**
 - Hard to interpret new features
 - Some information loss possible



Linear Discriminant Analysis (LDA) – Overview

- **Definition:**
Linear Discriminant Analysis (LDA) is a supervised **dimensionality reduction and classification** technique. It projects data onto a lower-dimensional space to **maximize class separability** while minimizing within-class variance.
- **Key Assumptions:**
 - Data follows a **Gaussian (Normal)** distribution.
 - **Equal covariance matrices** across classes.
 - Data is **linearly separable** (can be divided by a straight line/plane).
- **Working Principle:**
 - Compute **mean** and **scatter** (variance) for each class.
 - Calculate **between-class scatter** and **within-class scatter** matrices.
 - Find **eigenvectors and eigenvalues** from $S_w^{-1}S_b$
 - Select top components (directions) maximizing class separation.
 - Project data onto new LDA axes.
- **Goal:**
Maximize $J(v)$
 - → Large numerator (between-class distance), small denominator (within-class spread).
- **Extensions:**
 - **QDA:** Allows separate covariance per class.
 - **FDA:** Handles non-linear separability.
 - **RDA:** Adds regularization to avoid overfitting.

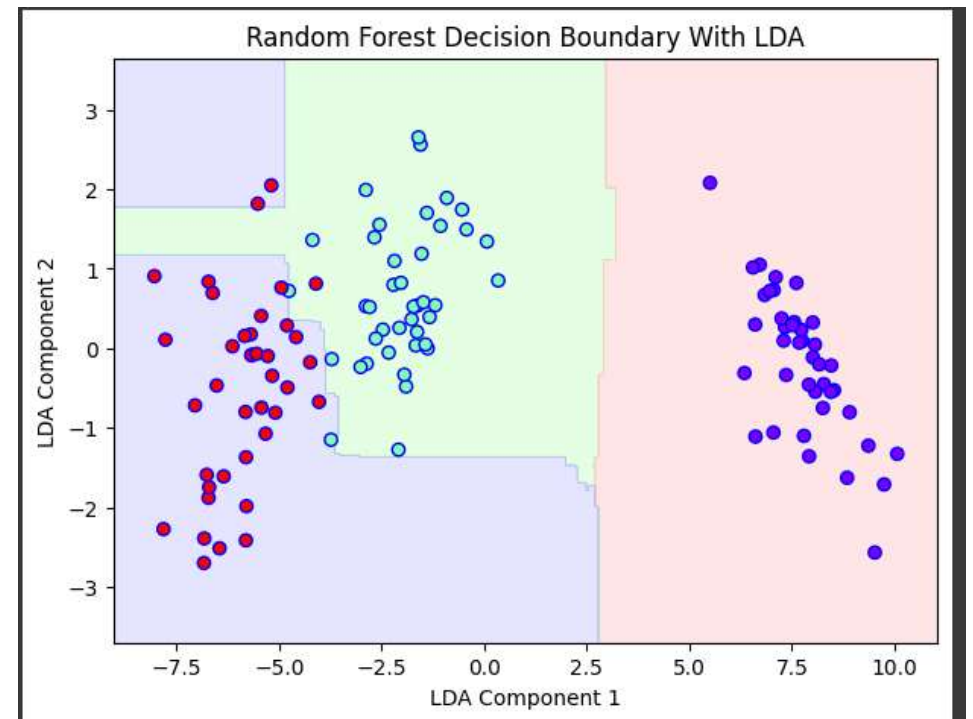


$$J(v) = \frac{|\mu_1 - \mu_2|^2}{s_1^2 + s_2^2}$$

LDA Implementation

- Overview

- **Decision Boundary Visualization:**
- LDA compresses data to 2D space.
- Improves **class separability** compared to raw features.
- **Advantages:**
 - Simple, fast, and interpretable.
 - Handles multicollinearity well.
 - Works well with small datasets.
- **Limitations:**
 - Assumes normality & equal covariance.
 - Fails on non-linear separability.
 - Can lose information in high dimensions.
- **Applications:**
 - Face recognition
 - Medical diagnosis
 - Customer segmentation



LDA vs PCA

Basis	PCA (Principal Component Analysis)	LDA (Linear Discriminant Analysis)
Type	Unsupervised technique	Supervised technique
Objective	Maximizes variance in the data	Maximizes class separability
Input Requirement	Does not require class labels	Requires class labels
Focus	Finds directions of maximum data spread	Finds directions that best separate classes
Computation Based On	Covariance matrix of data	Between-class and within-class scatter matrices
Dimensionality Reduction	Can reduce to any number of components	Can reduce to (C - 1) components, where C = number of classes
Use Case	Feature extraction and noise reduction	Classification and pattern recognition
Data Distribution	No assumption about data distribution	Assumes normal (Gaussian) distribution and equal covariance
Outcome	Components are uncorrelated	Components are discriminative
Example Application	Image compression, visualization	Face recognition, medical diagnosis

Feature Selection vs Feature Extraction

Aspect	Feature Selection	Feature Extraction
Purpose	Selects a subset of relevant existing features	Creates new features from original data
Dimensionality Reduction	Keeps original features but reduces their number	Transforms data into a lower-dimensional space
Techniques Used	Filter, Wrapper, Embedded methods	PCA, LDA, Kernel PCA, Autoencoders
Data Requirement	Needs domain knowledge for selecting features	Can be applied to raw, high-dimensional data
Output Features	Original features (interpretable)	New transformed features (less interpretable)
Advantages	Enhances interpretability, reduces overfitting	Captures hidden patterns, handles nonlinearity
Limitations	May discard useful data	May introduce redundancy or noise

Real Life Examples of Unsupervised Learning

- **Customer Segmentation:**
Businesses use unsupervised machine learning in Python to analyze user behavior, preferences, and purchase history. This helps identify distinct customer groups, allowing companies to design targeted marketing strategies, personalize offers, and improve product development. By segmenting customers based on data patterns, businesses can enhance engagement and satisfaction.
- **Anomaly Detection:**
Unsupervised learning is applied to detect unusual patterns or outliers in datasets. This technique helps identify irregularities, such as fraudulent transactions or abnormal system behavior, improving security and performance monitoring.
- **Recommendation Systems:**
Machine learning powers recommendation engines that personalize user experiences across platforms.
- Examples include **TED Talk**, **movie**, and **music** recommendations, where algorithms analyze preferences and suggest content aligned with user interests or emotions. These systems leverage data-driven insights to enhance engagement, satisfaction, and platform efficiency.

